

APP自动化测试



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌



Appium工具介绍及环境准备

黑马程序员:软件测试

主流的移动自动化测试工具



Robotium

- 是一款国外开源的Android自动化测试框架
- 适用平台：Android
- 支持语言：Java
- 不支持跨应用



macaca

- 是由阿里巴巴公司开源的一套自动化解决方案
- 适用平台：PC端、Android、iOS
- 支持语言：Java、Python、Node.js
- 支持跨应用



Appium

- 是一款国外开源的自动化测试工具
- 适用平台：Android、iOS
- 支持语言：Java、Javascript、PHP、Python、C#、Ruby等主流语言
- 支持跨应用
- 社区活跃、资料丰富

Appium的特点



Appium

- 开源;
- 支持**Native App、Web App、Hybird App**;
- 支持Android、iOS;
- Server也是跨平台的, 你可以使用Mac OS X、Windows或者Linux;
- 用Appium自动化测试不需要重新编译App;
- 支持很多语言来编写测试脚本, Java、Javascript、PHP、Python、C#、Ruby等主流语言;



Native App

原生应用, 使用Android或iOS的标准SDK编写的应用。

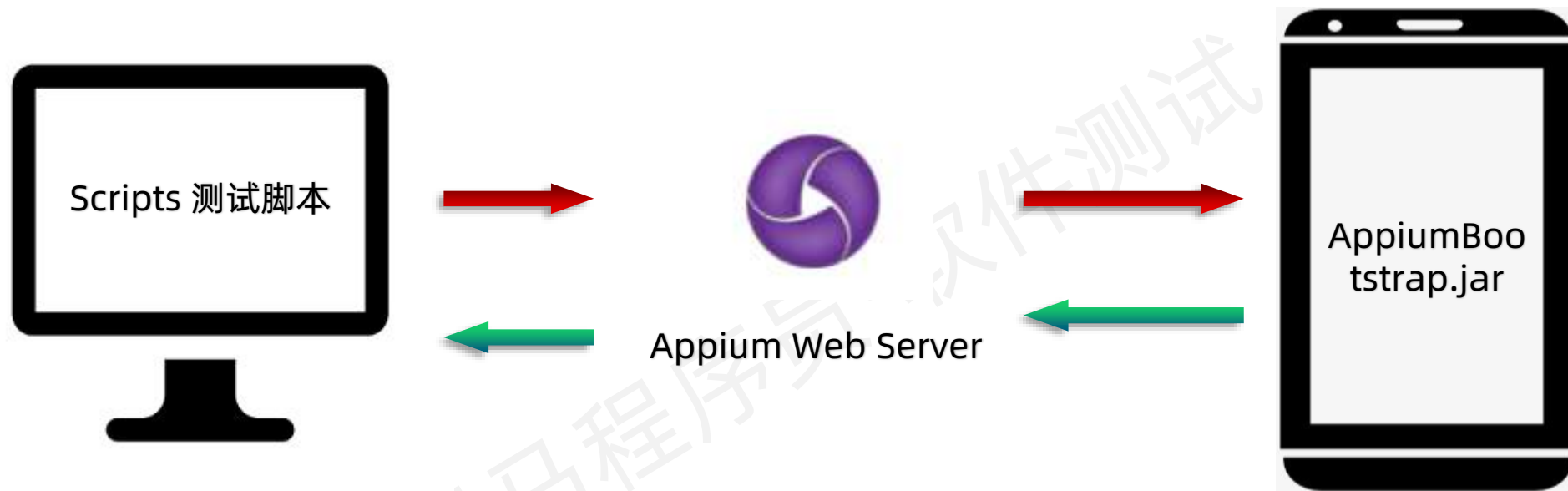
Web App

移动浏览器应用, 使用移动平台的浏览器访问的应用。

Hybird App

混合应用, 把一个基于webview实现的功能进行包装的应用。

Appium设计原理



- C/S架构，appium的核心是一个web服务器，提供了一套接口。他会接收客户端发送过来的命令，然后在移动设备上运行命令，最后把运行结果通过HTTP响应包返回给客户端。
- session，每个client连接到server以后都会创建一个session，自动化始终围绕一个session进行。

移动自动化测试环境步骤和核心组件介绍



- Java解释器
- SDK环境依赖于JDK
- 需要使用SDK中ADB工具连接手机
- 使用ADB工具进行部分手机测试工作
- 需要使用SDK中UIAutomatorViewer工具来查看APP元素信息
- Appium是识别脚本并转发到手机去执行脚本指令(web服务)中间桥梁
- 实际测试APP所在的容器，可以是真机也可以是模拟器

安装JDK



版本: JDK1.8

地址: <https://www.oracle.com/java/technologies/javase/javase-jdk8-downloads.html>

安装步骤

- 1、下载JDK1.8安装包，区分操作系统和系统位数；
- 2、根据指引安装JDK，尽量选择非中文目录；
- 3、配置环境变量：
JAVA_HOME=C:\Program Files\Java\jdk1.8.0_151 **(这里为JDK安装目录)**
在Path中添加:%JAVA_HOME%\bin;

验证安装结果

打开命令行窗口，输入java -version，校验命令能否正常执行以及版本信息是否一致

```
Microsoft Windows [版本 10.0.18363.1379]
(c) 2019 Microsoft Corporation. 保留所有权利。

C:\>java -version
java version "1.8.0_161"
Java(TM) SE Runtime Environment (build 1.8.0_161-b12)
Java HotSpot(TM) Client VM (build 25.161-b12, mixed mode)
```

安装Android SDK



地址: <https://android-sdk.en.softonic.com/>

介绍: 手机应用开发通用软件

安装步骤

- 1、下载安装包，并解压，尽量不要放置到中文目录；
- 2、解压之后的安装包只包含基本的SDK工具，它不包含Android平台或任何第三方库。需要使用SDK Manager安装所需要的工具；
- 3、配置环境变量：
 `ANDROID_HOME=D:\Android\sdk` **(这里为SDK安装目录)**
 在Path中添加:`%ANDROID_HOME%\tools;%ANDROID_HOME%\platform-tools;`

验证安装结果

打开命令行窗口，输入`adb -version`，校验命令能否正常执行以及版本信息是否一致

```
Microsoft Windows [版本 10.0.18363.1379]
(c) 2019 Microsoft Corporation。保留所有权利。

C: >adb --version
Android Debug Bridge version 1.0.39
Version 0.0.1-4500957
Installed as D:\android-sdk\platform-tools\adb.exe
```


安装Appium和模拟器



版本: 1.22.3

地址: <https://github.com/appium/appium-desktop/releases>

安装步骤

下载安装包后直接以管理员方式安装，下一步到最后即可



版本: 3.9.2稳定版本

地址: <https://mumu.163.com/>

安装步骤

下载安装包后直接以管理员方式安装，下一步到最后即可

安装Appium-python库

Appium-python库

说明： 编写测试脚本时所需要使用api包

安装步骤

直接打开CMD，通过PIP命令在线下载

```
pip install Appium-Python-Client
```

验证安装结果

成功 输入pip list，在列表清单中如果可以找到安装Appium-Python-Client则表示

案例

启动商品APP首页



示例脚本

```
import time
from appium import webdriver
from appium.options.android import UiAutomator2Options
des_caps = {
    "platformName": "Android",
    "deviceName": "mumu",
    "platformVersion": "12",
    "appPackage": "com.tpshop.malls",
    "appActivity": ".SPMainActivity"
}
opt = UiAutomator2Options().load_capabilities(des_caps)
driver = webdriver.Remote("http://127.0.0.1:4723/wd/hub",
    options=opt)

time.sleep(5)

driver.quit()
```



测试环境准备:

- 打开模拟器
- 启动Appium服务器
- 获取要测试APP的包名/界面名

- Windows: `adb shell dumpsys window | findstr mCurrentFocus`
- Mac/Linux: `adb shell dumpsys window | grep mCurrentFocus`

初始化配置项介绍

```
des_caps = {  
    "platformName": "Android",  
    "deviceName": "mumu",  
    "platformVersion": "12",  
    "appPackage": "com.tpshop.malls",  
    "appActivity": ".SPMainActivity"  
}
```



des_caps字典对象的每个键名分别表示什么?

不同手机型号是否启动参数都是一样的?

platformName: 需要连接的手机的**平台**(不限制大小写)

devicesName: 需要连接的手机的**设备号**(andoird平台下, 可以随便写, 但是不能不写)

platformVersion: 需要连接的手机的**版本号**(比如 12 的版本可以填写 12.0.1 或 12.0 或 12, 以此类推)

appPackage: 需要启动的程序的**包名**

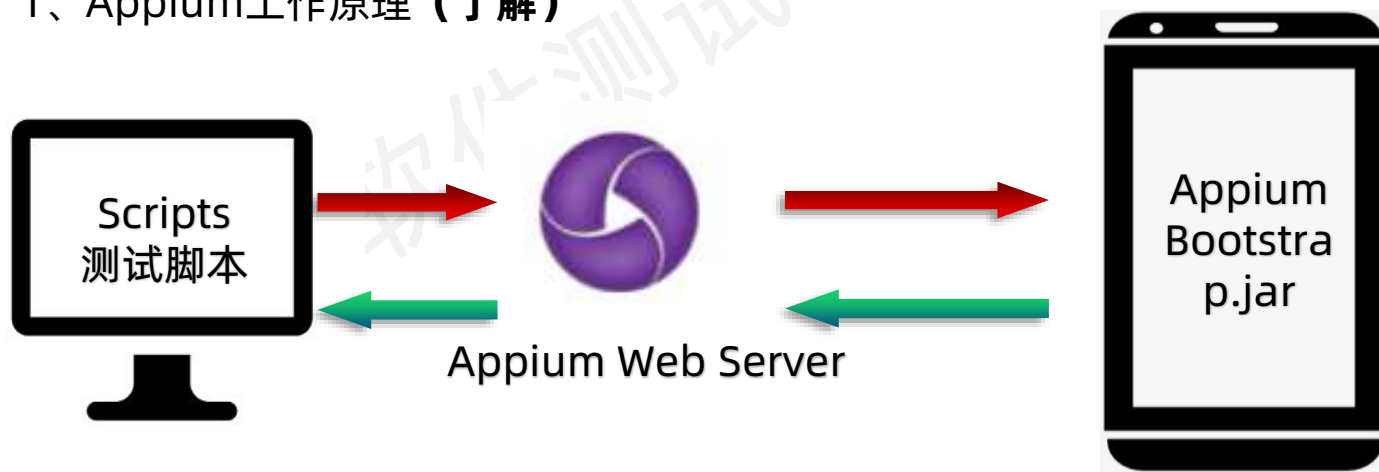
appActivity: 需要启动的程序的**界面名**

```
driver = webdriver.Remote( command_executor: "http://127.0.0.1:4723/wd/hub", options=opt)
```

Remote第一个参数: 连接到appium的接口地址, appium服务器部署在本机, 4723为应用端口, wd/hub为接口地址

总结

1、Appium工作原理（了解）



2、APP自动化环境搭建（了解）

安装JDK

安装Android SDK

安装Appium

安装模拟器

安装Appium-python

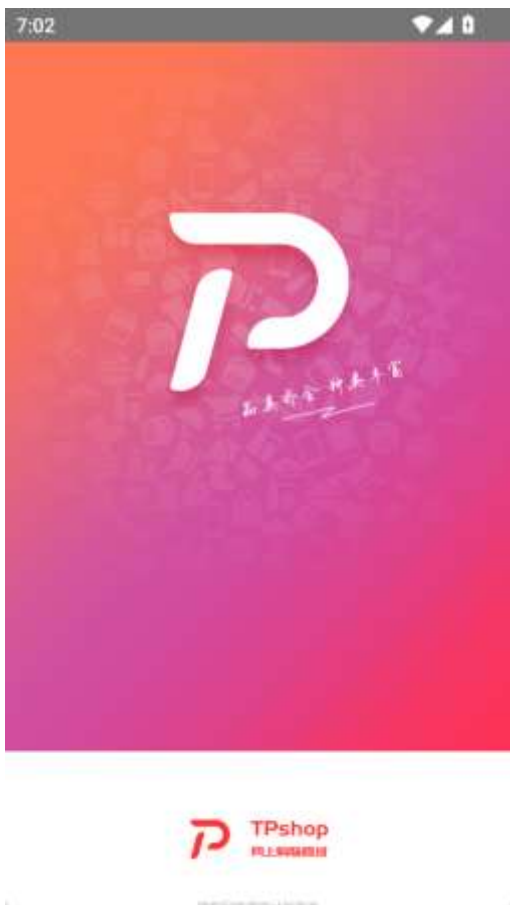


入门案例

黑马程序员软件测试

案例

启动商城app登录



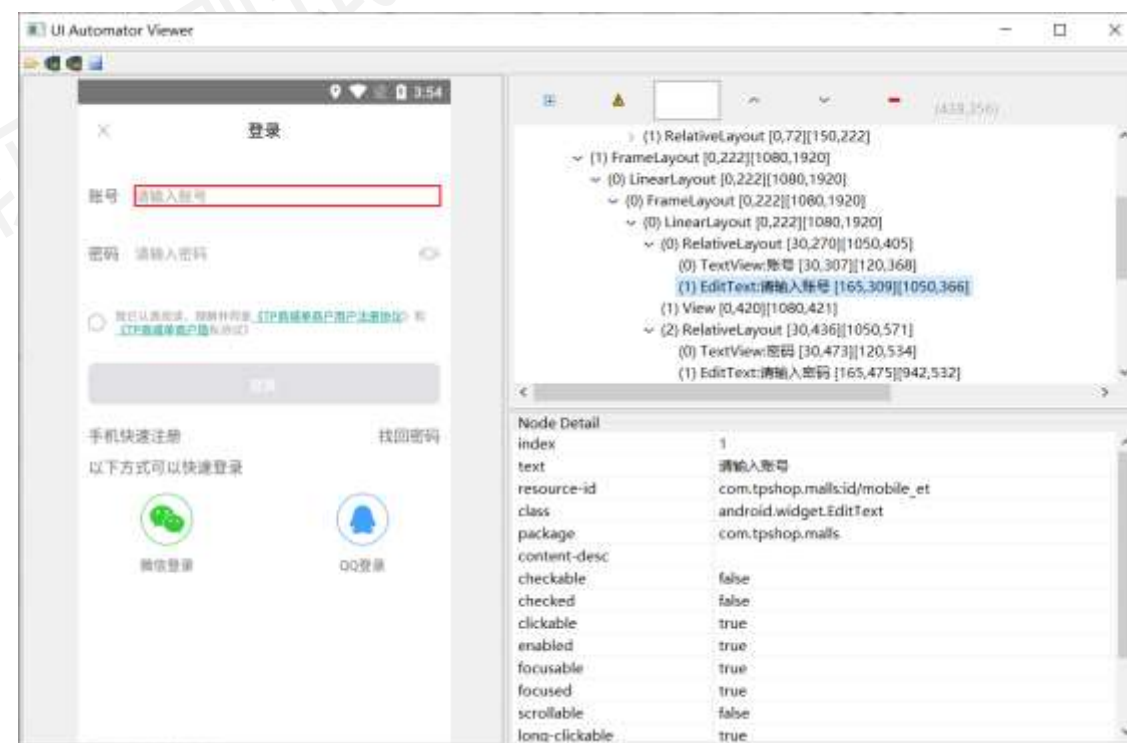
UIAutomatorViewer查看元素信息

UIAutomatorViewer

定位元素的时候必须根据元素的相关特征来进行定位，而 UIAutomatorViewer 就是用来获取元素特征的。

查看APP元素信息步骤

- 1. 进入SDK目录下的目录：
mac 在 tools/bin 目录下，打开 uiautomatorviewer
windows 在 tools 目录下，打uiautomatorviewer.bat
- 2. 电脑连接真机或打开android模拟器
- 3. 启动待测试app
- 4. 点击 uiautomatorviewer 的左上角 Device Screenshot （从左数第二个按钮）
- 5. 点击希望查看的控件
- 6. 查看右下角 Node Detail 相关信息



注意

在打开工具后会有一个黑屏窗口，不能关闭！！

UIAutomatorViewer使用常见问题



问题一：UiAutomatorViewer闪退

- 通常是由于jdk版本不兼容导致，一般情况下使用jdk1.8

问题二：点击捕获界面元素信息报错

- 通常是因为**UiAutomatorViewer**连接不到模拟器/手机导致，只需要重置adb服务即可
 - adb kill-server # 关闭adb服务
 - adb start-server # 启动adb服务



一定要养成独立解决问题的能力!!!!

定位单个元素



通过UiAutomatorViewer可以获取到元素信息，appium提供根据这些信息找到具体元素方法

Node Detail	
index	0
text	WLAN
resource-id	com.android.settings:id/title
class	android.widget.TextView
package	com.android.settings
content-desc	
checkable	false
checked	false
clickable	false
enabled	true
focusable	false
focused	false
scrollable	false
long-clickable	false
password	false
selected	false
bounds	[216,457][368,530]

ID定位

```
driver.find_element(By.ID, resource-id属性值)
```

class定位

```
driver.find_element(By.CLASS_NAME, class属性值)
```

xpath定位

```
driver.find_element(By.XPATH, xpath表达式)
```

注意

- 当定位到多个符合条件的元素时，默认返回第一个

模拟操作-点击、输入、清空

模拟点击

```
element.click()
```

模拟输入

```
element.send_keys(value)
```

清除文本

```
element.clear()
```

注意

默认输入中文无效，但不会报错，需要在 " 前置代码 " 中增加两个参数：

```
des_caps['unicodeKeyboard'] = True  
des_caps['resetKeyboard'] = True
```

案例

自动登录



3:52

×

登录

账号 请输入账号

密码 请输入密码

☐ 我已认真阅读、理解并同意《TP商城单商户注册协议》和《TP商城单商户隐私协议》

登录

手机快速注册 找回密码

以下方式可以快速登录

微信登录 QQ登录

示例脚本

```
# 执行操作
# 1、输入用户名
driver.find_element(By.ID,
"com.tpshop.malls:id/mobile_et").send_keys("13488888888")
# 2、输入密码
driver.find_element(By.ID, "com.tpshop.malls:id/pwd_et").send_keys("123456")
# 3、勾选协议
driver.find_element(By.ID, "com.tpshop.malls:id/agree_btn").click()
# 4、点击登录
driver.find_element(By.ID, "com.tpshop.malls:id/login_tv").click()
```

总结

1、如何定位单个元素？

ID定位

`driver.find_element(By.ID, resource-id属性值)`

class定位

`driver.find_element(By.CLASS_NAME, class属性值)`

xpath定位

`driver.find_element(By.XPATH, xpath表达式)`

2、元素操作常见方法？

模拟点击

`element.click()`

模拟输入

`element.send_keys(value)`

清除文本

`element.clear()`

03

Appium-基础Api

- app基础操作：应用跳转、获取包名和界面名、关闭app和驱动对象、安装卸载和判断是否安装、置于后台
- 元素定位：定位单个元素、定位一组元素
- 元素操作：点击、输入、清空、获取元素信息、获取属性值信息
- 滑动和拖拽

学习目标

- 能灵活运用Appium-基础api实现对于app应用的模拟测试过程脚本编写

app基础操作-应用跳转



方法说明

```
driver.start_activity(appPackage, appActivity)
```

appPackage: 要打开的程序的包名

appActivity: 要打开的程序的界面名

示例和练习

打开《设置》应用程序，等待3秒后跳转到《tpshop》应用程序

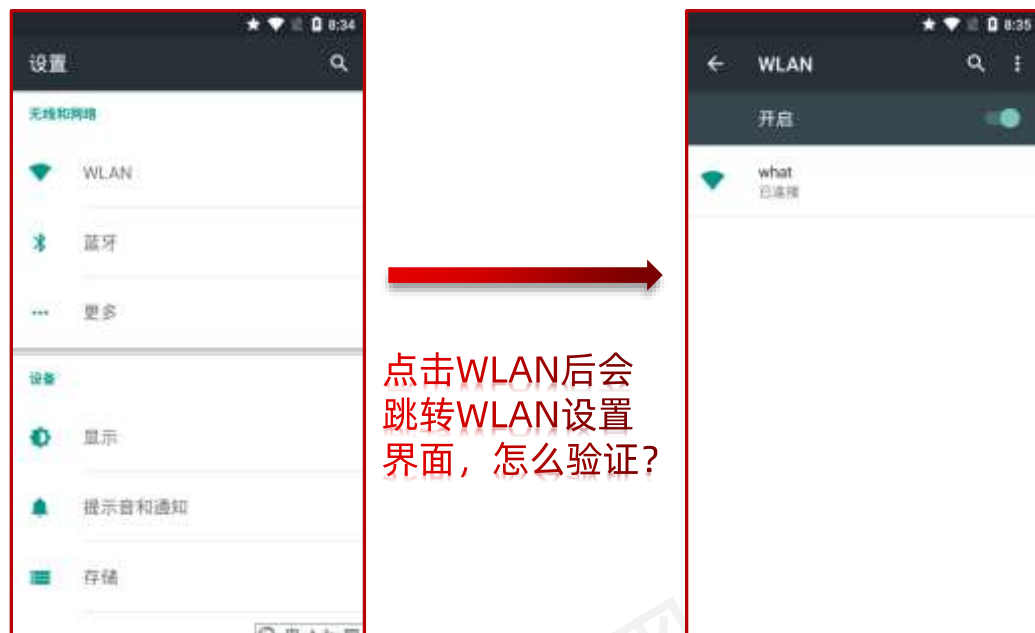
```
import time
from appium import webdriver
from appium.options.android import UiAutomator2Options
des_caps = {
    "platformName": "Android",
    "deviceName": "xxx",
    "platformVersion": "12",
    "appPackage": "com.android.settings",
    "appActivity": "com.android.settings.Settings"
}
opt = UiAutomator2Options().load_capabilities(des_caps)
driver = webdriver.Remote("http://127.0.0.1:4723/wd/hub", options=opt)

time.sleep(3)
driver.start_activity("com.tpshop.malls", "com.tpshop.malls.SplashActivity")

time.sleep(5)

driver.quit()
```

app基础操作-获取包名和界面名



属性说明

获取包名
driver.**current_package**

获取界面名
driver.**current_activity**

示例和练习

打开《设置》应用程序，等待3秒获取界面名和包名

```
import time
from appium import webdriver
from appium.options.android import UiAutomator2Options

caps = {
    "platformName": "Android", # 移动端系统平台
    "platformVersion": "12", # 平台对应版本
    "deviceName": "mumu", # 设备名称（可以随便写）
    "appPackage": "com.android.settings", # 包名
    "appActivity": ".Settings" # Activity页面名
}

opt = UiAutomator2Options().load_capabilities(caps)
driver = webdriver.Remote("http://127.0.0.1:4723/wd/hub", options=opt)
time.sleep(3)

print(driver.current_package)
print(driver.current_activity)

time.sleep(5)

driver.quit()
```


app基础操作-关闭驱动和app



有的时候我们需要关闭某个应用程序后，再打开新的应用。那么如何关闭应用程序呢？

方法说明

关闭当前操作的app，不会关闭驱动对象
`driver.close_app()`

关闭驱动
`driver.quit()`

注意

- 关闭app后还能继续操作手机其它应用；
- 关闭驱动，可以理解为锁屏，不能在进行任何操作。

示例和练习

打开《设置》，使用 `close_app()` 方法关闭，再尝试使用 `quit()` 方法，最后打印当前程序的包名，观察区别

```
import time
from appium import webdriver

caps = {
    "platformName": "Android",
    "deviceName": "xxx",
    "platformVersion": "7.1.2",
    "appPackage": "com.android.settings",
    "appActivity": "com.android.settings.Settings"
}

driver = webdriver.Remote("http://127.0.0.1:4723/wd/hub", caps)
time.sleep(3)
driver.close_app()
print(driver.current_package)
print(driver.current_activity)

time.sleep(5)
driver.quit()
print(driver.current_package)
print(driver.current_activity)
```

app基础操作-安装和卸载以及是否安装 app



场景：部分应用管理平台在安装app后，会自动进行检测，如已安装了则会显示【启动】

方法说明

安装app, app_path为安装文件完整路径名
driver.**install_app**(app_path)

卸载app, app_id为app包名
driver.**remove_app**(app_id)

判断app是否安装, app_id为app包名
driver.**is_app_installed**(app_id)

示例和练习

如果《tpshop》已经安装，则卸载《tpshop》，如果没有则安装

```
import time
from appium import webdriver

caps = {
    "platformName": "Android",
    "deviceName": "xxx",
    "platformVersion": "7.1.2",
    "appPackage": "com.android.settings",
    "appActivity": "com.android.settings.Settings"
}

driver = webdriver.Remote("http://127.0.0.1:4723/wd/hub", caps)

time.sleep(3)
if driver.is_app_installed("com.tpshop.malls"):
    driver.remove_app("com.tpshop.malls")
else:
    driver.install_app(r"d:\soubaoShopMobile-release.apk")

time.sleep(5)
driver.quit()
```

app基础操作-置于后台



登陆后置于后台一段时间后，这类安全系数高的APP再重新打开使用时会要求重新登陆！

方法说明

置于后台，seconds表示秒数
driver.**background_app(seconds)**

示例和练习

打开《设置》应用，进入后台 5 秒，再回到前台

```
import time
from appium import webdriver
```

```
caps = {
    "platformName": "Android",
    "deviceName": "xxx",
    "platformVersion": "7.1.2",
    "appPackage": "com.android.settings",
    "appActivity": "com.android.settings.Settings"
}
```

```
driver = webdriver.Remote("http://127.0.0.1:4723/wd/hub", caps)
driver.background_app(5)
```

```
time.sleep(5)
driver.quit()
```



小结

- 跳转应用: driver.start_activity(appPackage, appActivity)
- 获取包名: driver.current_package
- 获取界面名: driver.current_activity
- 关闭APP: driver.close_app()
- 关闭驱动: driver.quit()
- 安装app: driver.install_app(app_path)
- 卸载app: driver.remove_app(app_id)
- 判断app是否安装: driver.is_app_installed(app_id)
- 置于后台: background_app(seconds)

定位一组元素



要获取搜索到的商品的名称，一个一个的定位然后在获取文本么？



ID定位

`driver.find_elements(By.ID, resource-id属性值)`

class定位

`driver.find_elements(By.CLASS_NAME, class属性值)`

xpath定位

`driver.find_elements(By.XPATH, xpath表达式)`

注意

- 使用定位一组元素时，返回的数据是列表

练习

根据要求完成下面代码步骤

需求：打开手机《设置》应用，完成下面的步骤：

- ①.使用ID定位，获取所有 resource-id 为 " android:id/title " 的元素，并打印其文字内容
- ②.使用CLASS定位，获取所有class 为 " android.widget.TextView " 的元素，并打印其文字内容
- ③.使用XPATH定位，获取所有包含 " 设 " 的元素，并打印其文字内容
- ④.等待3s，关闭app



利用元素属性定位元素

```
//*[@属性名='属性值']
```

利用局部属性值定位元素

```
//*[contains(@attribute,'局部属性值')]
```

获取元素信息-文本、位置和大小

获取文本

element.text



测试搜索商品结果的显示商品名称是否正确就需要获取文本



获取位置

element.location

获取大小

element.size



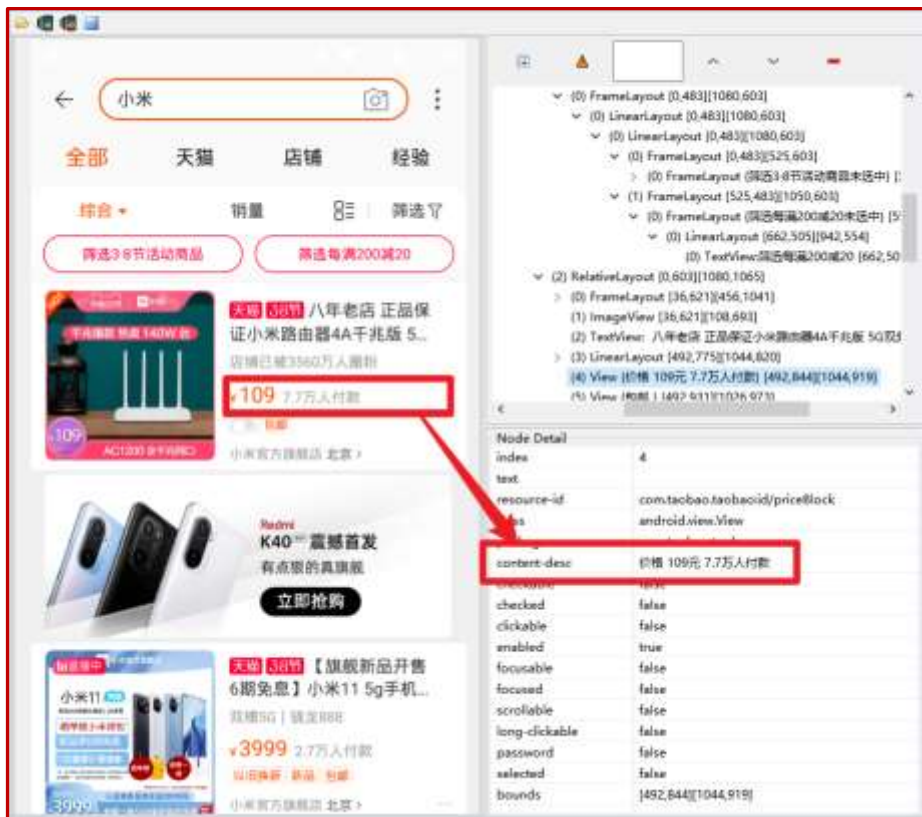
操作新闻顶部滑动时需要按住坐标进行滑动，就需要根据获取频道区域元素位置和大小来计算出滑动坐标！



获取元素信息-获取元素属性值



测试商品显示金额是否正确就需要获取属性值



获取属性值

element.get_attribute(属性名)

Node Detail	
index	2
text	八年老店 正品保证小米路由器4A千兆版 5...
resource-id	com.taobao.taobao:id/title
class	android.widget.TextView
package	com.taobao.taobao
content-desc	
checkable	false
checked	false
clickable	false
enabled	true

注意：获取属性值对应的属性名和实际UIAutomatorViewer显示的**不一定一致**！！！！

获取resource-id

resourceId

获取class

className

获取text

text

案例

根据要求完成下面代码步骤

需求：打开手机《设置》应用，完成下面的步骤：

- ①.使用ID定位，获取所有 resource-id 为 " android:id/title " 的元素，并打印其文字内容，位置和大小
- ②.使用 get_attribute 获取这些元素的 resource-id、class、text属性值

获取resource-id

resourceId

获取class

className

获取text

text

```
import time
from appium import webdriver
caps = {
    "platformName": "Android",
    "deviceName": "xxx",
    "platformVersion": "7.1.2",
    "appPackage": "com.android.settings",
    "appActivity": "com.android.settings.Settings"
}
elements = driver.find_elements(By.ID, "android:id/title")
driver = webdriver.Remote("http://127.0.0.1:4723/wd/hub", caps)
for i in elements:
    print(f"元素文本信息：{i.text}，元素位置信息：{i.location}，元素的大小：{i.size}")
for i in elements:
    print(f"元素的resource-id属性值为：{i.get_attribute('resourceId')}，元素的class属性值为：{i.get_attribute('className')}，元素的文本属性为：{i.get_attribute('text')}")
time.sleep(5)
driver.quit()
```

04

Appium-手机操作

- 常用手机操作Api：获取分辨率、手机截图、获取和设置网络、发送键到设备、打开和关闭通知栏

学习目标

- 在实现自动化测试脚本过程中对于手机本身的操作能使用api来实现

手机操作Api-获取分辨率和截图



淘宝商品通过坐标点翻页，
如换了分辨率还是原坐标么？

通过获取手机分辨率来动态计算滑动的起止坐标点

实现方法

- 获取手机分辨率。
`driver.get_window_size()`

有些自动化的操作可能没有反应，但并不报错。此时我们就可以将操作过后的关键情况，截图留存。后期也可以根据图片发现问题

实现方法

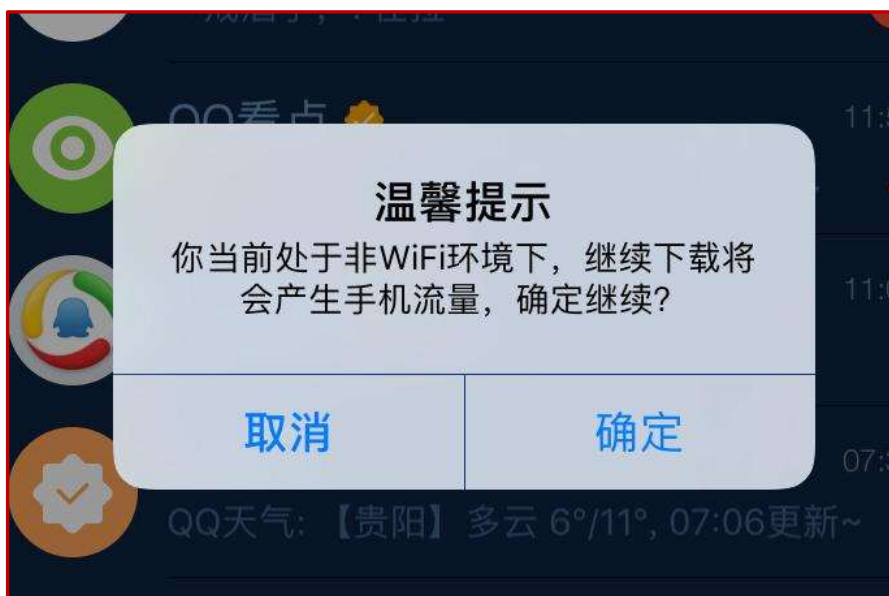
- 获取手机截图。
`get_screenshot_as_file(filename)`



手机操作Api-获取和设置手机网络



下面的场景是否遇到过，那么基于APP是否需要进行测试？



手机网络对照图

Possible values:			
Value (Alias)	Data	Wifi	Airplane Mode
0 (None)	0	0	0
1 (Airplane Mode)	0	0	1
2 (Wifi only)	0	1	0
4 (Data only)	1	0	0
6 (All network on)	1	1	0

实现方法

- 获取手机网络。
`driver.network_connection`
 - 设置手机网络。
`driver.set_network_connection(connectionType)`
- `connectionType`: 表示网络设置类型



练习：打印当前模拟器网络类型，并设置飞行模式。

手机操作Api-发送键到设备



需求：《王者荣耀》APP在按HOME键时至于后台？按两次返回键弹出确认退出提示窗口。

实现方法

- 发送键到设备。
`driver.press_keycode(keycode)`
`keycode`：表示手机设备的默认键码

参考键码表：

<https://blog.csdn.net/feizhixuan46789/article/details/16801429>



练习：点击三次音量加，再点击返回，再点击两次音量减。

手机操作Api-操作通知栏

《网易新闻》APP有新闻推送，
需要进行测试需要先打开通知栏



实现方法

- 打开通知栏。
`driver.open_notifications()`

注意

appium官方并没有为我们提供关闭通知的api，那么现实生活中怎么关闭，就怎样操作就行，比如，手指从下往上滑动，或者，按返回键



练习：打开通知栏，两秒后，关闭通知栏。



Appium-其他Api

- 滑动、拖拽

swipe滑动



在做测试的时候，有些按钮是需要滑动几次屏幕后才会出现，此时，我们需要使用代码来模拟手指的滑动



Swipe滑动

从一个坐标位置滑动到另一个坐标位置，只能是两个点之间的滑动

方法

```
driver.swipe(start_x, start_y, end_x, end_y, duration=None)
```

start_x: 起点X轴坐标

start_y: 起点Y轴坐标

end_x: 终点X轴坐标

end_y: 终点Y轴坐标

duration: 滑动这个操作一共持续的时间长度，单位：ms



如果起始坐标和结束坐标写死了，更换不同分辨率的手机后，是不是可滑动不了呢？

起始坐标和结束坐标通过获取滑动区域元素位置和大小来动态计算。

注意

swipe适合多次滑动，duration时间越长惯性越大



根据要求完成下面代码步骤

练习：打开手机《设置》应用，完成下面的步骤：

①.模拟手指从（100, 1200），滑动到（100, 300）的位置。

黑马程序员·软件测试

scroll滑动

Scroll滑动

从一个元素滑动到另一个元素，直到页面自动停止

方法

`driver.scroll(origin_el, destination_el)`

origin_el: 滑动开始的元素

destination_el: 滑动结束的元素

注意

- scroll滑动是两个元素之间的滑动只适合滑动一次的操作
- 惯性很大

练一练

需求：打开《设置》应用，从“更多”滑动到“更WLAN”

drag_and_drop拖拽

Scroll滑动

从一个元素滑动到另一个元素，第二个元素替代第一个元素原本屏幕上的位置

方法

```
driver.drag_and_drop(origin_el, destination_el)
```

origin_el: 滑动开始的元素

destination_el: 滑动结束的元素

注意

- scroll滑动是两个元素之间的滑动只适合滑动一次的操作
- 无惯性



练一练

需求：打开《设置》应用，从“更多”拖拽到“WLAN”



传智教育旗下高端IT教育品牌